

School of Electronic Engineering
and Computer Science

Final Report

Computer Science

Lifting Log

an Android weightlifting tracker

Student:

Mohammed Hamza Huda

Supervisor:

Nafi Ahmad

Final Year
Undergraduate Project 2025/26

Date: 05/05/2026

Abstract

Although there are many weightlifting tracking applications in the market, the largest applications with the largest user bases lack key features that would greatly improve the user experience for the average user, are bloated with unnecessary features, and paywall features that are computationally inexpensive.

The objective of this project was to develop a weightlifting tracker app that uses user lifting data to help them achieve their lifting goals. The app will provide features like automatic progressive overload and estimated strength gain timelines based on past performance, with the goal of making the app feel like an “Invisible Coach”, such that when you go to the gym, you can focus on lifting without having to stress over what you should be doing or if you are on the right track or not.

The resulting Android application “Lifting-Log” was developed using a React Native frontend and a Django REST Framework backend. It successfully integrates “Invisible Coach” features, from using the Brzycki formula for 1RM data normalisation to use in mathematical regression models (linear and logarithmic) to forecast strength gains, to automated progressive overload. Overall, the application successfully fulfils the core aims of the project so the user can focus on their workout.

C contents

Chapter 1:	Introduction	7
1.1	Background	7
1.2	Problem Statement	7
1.3	Aim	7
1.4	Objectives	7
1.5	Research Questions	8
1.6	Report Structure	8
Chapter 2:	Research.....	9
2.1	About Weightlifting.....	9
2.1.1	Sets and Reps	9
2.1.2	Progressive Overload	9
2.1.3	One-repetition Maximum (1RM).....	9
2.1.4	Estimated 1RM	9
2.1.5	Predicting Future Strength Gains	10
2.2	Existing Applications	11
2.2.1	Hevy	11
2.2.2	JEFIT	12
2.2.3	Strong	12
2.3	Summary.....	13
Chapter 3:	Requirements Analysis.....	14
3.1	Functional Requirements.....	14
3.2	Non-Functional Requirements	14
Chapter 4:	Design.....	15
4.1	Use Cases.....	15
4.2	Development Environment and Technologies	16
4.2.1	Mobile Client (Frontend).....	16

4.2.2	Server (Backend)	16
4.3	System Architecture	17
4.4	Database	18
4.5	Core Algorithms	19
4.5.1	1RM Calculation	19
4.5.2	Strength Prediction	19
4.5.3	Automatic Progressive Overload	19
Chapter 5:	Backend Implementation	20
5.1	Workspace Setup	20
5.2	Models	21
5.2.1	Model Fields	21
5.2.2	Model Methods	22
5.3	Custom Permissions	23
5.4	Views and API Endpoints	23
5.4.1	QuerySet Filtering	24
5.4.2	Custom API Actions	24
5.5	Serializers	24
5.5.1	Nested Serializers	25
5.5.2	Connecting Custom Methods	25
5.6	Authentication	25
5.6.1	JWT Verification	26
5.6.2	Local User Synchronisation	26
Chapter 6:	Frontend Implementation	27
6.1	User Authentication and Management	28
6.1.1	Supabase Client Initialisation	28
6.1.2	Auth State Management	28
6.1.3	Sign In Screen	28
6.2	UI Architecture	29
6.2.1	NativeWind	29
6.2.2	React Native Reusables	29

6.3	API Service Layer	29
6.3.1	Request Interceptor	29
6.3.2	API Functions	30
6.4	Local Database for Custom Exercises	30
6.5	Templates	31
6.6	Workouts	33
6.7	Progress	34
6.8	Account Tab	36
Chapter 7:	Testing	37
7.1	Automated Backend Tests	37
7.1.1	Model Tests	37
7.1.2	API and Integration Tests	37
7.1.3	Permissions Tests	37
7.1.4	Test Table.....	38
7.2	Manual Frontend Tests	38
7.3	User Tests	39
7.3.1	Methodology	39
7.3.2	Feedback.....	39
Chapter 8:	Evaluation	40
8.1	Requirements	40
8.2	Resolution of Research Questions	40
8.2.1	How can historical lifting performance be mathematically modelled to predict future strength gains accounting for new and experienced lifters?	40
8.2.2	How much does automatic progressive overload improve a user's workout?	40
8.2.3	What features of market available weightlifting trackers are unnecessary for an average user?	40
8.2.4	How useful is strength prediction for motivating the end user to keep lifting?	40
8.3	Limitations	41
Chapter 9:	Conclusion	42
9.1	Achievements	42
9.2	Challenges	42

9.3	Future Work	43
9.4	Summary	43
	References	44
	Appendix A – Gen AI Tools Accountability Statement.....	45
	Appendix B – Project Time Plan	46

Chapter 1: Introduction

1.1 Background

Weightlifting as a hobby and a means of getting in shape is universal, recognisable to people from all parts of the globe. I developed an interest in weightlifting in my teens and, from then, coached friends and family on how to lift weights, introducing them to the gym, teaching them proper form, and showing them how to progressively overload. Since I introduced my first friend to the gym, I've been passing along an excel spreadsheet I made to track my progress. This sheet has served me well over the years, however other people find it clunky, tedious, unintuitive – it is geared towards a less casual lifter and requires the user to analyse their past progress to determine what to do during their next session. This is where the idea of building a weightlifting tracker app came from.

1.2 Problem Statement

New weightlifters often fail to progress because they don't track their lifting performance nor know how to interpret it, leading to bad weight selection and the inability to set realistic goal timelines, something usually only an experienced personal trainer could provide.

1.3 Aim

The aim of this project is to design and build a weightlifting tracker app that automatically tells users what weight to lift during a session based on their past performance and uses that past performance to predict (linearly for beginners or logarithmically otherwise) the trajectory of future strength gains.

The target audience are new lifters, and I hope the app serves as an "Invisible Coach" that knows all your lifting performance, so that when you go to lift weights you don't have to do any thinking.

To achieve this, I plan on learning how to develop an Android application, as I've never done so before, and to use my limited knowledge of web development to aid me in this process.

1.4 Objectives

- Evaluate the most popular, existing solutions on the market

- Interview friends and family to see what they would want out of a weightlifting app

- Research how past lifting data can be used to predict future strength growth trajectories

- Develop the core CRUD Android application, with workout routines and workout logging

Integrate the “Invisible Coach” features, including, but not limited to, automatic progressive overload and goal weight timeline prediction based on past performance

To test and evaluate the functionality and success of the application based on the aims

1.5 Research Questions

These questions were explored throughout the research, development and testing of the application.

1. How can historical lifting performance be mathematically modelled to predict future strength gains accounting for new and experienced lifters?
2. How much does automatic progressive overload improve a user’s workout?
3. What features of market available weightlifting trackers are unnecessary for an average user?
4. How useful is strength prediction for motivating the end user to keep lifting?

1.6 Report Structure

Chapter two is an exploration of the fundamental concepts of weightlifting, predicting future strength gains, and an evaluation of existing Play Store applications.

Chapter three presents functional and non-functional requirements based on the research phase’s findings.

Chapter four is an outline of the system architecture, development choices, use cases, database architecture, and the core algorithms to be used.

Chapter 5 and 6 provide an in depth look at technical implementation of the Django REST backend and React Native frontend.

Chapter 7 is a discussion on how the final application was tested, with automated tests for the backend and manually tested on the frontend, on top of user testing.

Chapter 8 is an evaluation on the final application’s success against the aims and objectives, a look towards more work that could be done, and a conclusion of the project.

Chapter 2: Research

Research into weightlifting and existing applications provides insight on the necessary scope of the application and will help inform what the requirements should consist of so that the aims can be achieved.

2.1 About Weightlifting

Weightlifting is the act of, as the name suggests, lifting weights. There are many uses for resistance training; however, this app is focused on helping the user lift weights for the purpose of strength gains and hypertrophy (muscle growth).

What follows are explanations of terminology that will be used throughout the report.

2.1.1 Sets and Reps

Lifting weights is ordinarily done through sets and reps. A singular repetition (rep) of an exercise consists of the concentric, when the muscles contract, and the eccentric, where the muscles lengthen with tension to control the weight. If we take the bench press, a popular exercise where the lifter lays flat on their back on a bench and lowers and lift a barbell, the concentric portion would when the bar is being pushed up, and the eccentric portion would be when the bar is being lowered down. The concentric and eccentric movement combined, just once, consists of one rep of an exercise.

Usually, lifters perform multiple reps simultaneously, before setting down the weight. That block of repeated reps is what a set is called, for example, a lifter might do 8 reps of a bench press before re-racking the weight, that would be one set, the number of reps performed would be dependent on their training program. After one set is complete, before beginning another set, the lifters rest for a period, before beginning another set, and they repeat as many times as their program suggests, which is generally in the range of 1-5 sets.

2.1.2 Progressive Overload

Progressive overload, in weightlifting, is when the demand on the muscles is gradually increased over time to stimulate strength gains, muscle growth, and fitness. Some examples include increasing reps, weight, volume, or intensity.

2.1.3 One-repetition Maximum (1RM)

A 1RM (one-repetition maximum) is the maximum weight a person can lift for a single complete repetition. It is the most accurate way to compare peak strength between people, and as such is used in competitive weightlifting for scoring.

2.1.4 Estimated 1RM

For the average person's goals, testing your 1RM is usually unnecessary, as pushing your body to its limit poses an increased risk of injury and causes significant physical fatigue. However, a person's 1RM is still the best way to measure their strength gains overtime. To solve this dilemma, through 1RM calculation formulae, we can take sets performed with multiple reps, normalising it to 1 rep, and calculate an estimated 1RM.

2.1.5 Predicting Future Strength Gains

By analysing a user's workout history, specifically their estimated 1RM trend over time, it is possible to extrapolate future progress using mathematical models; the two primary models used for this are linear and logarithmic (Steele et al., 2022).

Linear progression assumes that strength increases at a constant rate over time, this is most seen in novice lifters – people new to training (Aslam, Habyarimana and Bin, 2025). In this case, if the history shows an increase in 2.5kg every week, the linear model will predict they will continue to do the same and calculate when they would hit their goal based on such. The limitation, of course, is that linear progression is finite, as a lifter gains experience, the harder it becomes to gain strength, this is accounted for by a logarithmic model.

The logarithmic model is more suited to intermediate and advanced lifters, a logarithmic model accounts for the curve of rapid initial gains which then taper off over time, flattening out. It manages expectations as strength gains are possible yet slower than when a novice (Aslam, Habyarimana and Bin, 2025).

By determining what phase a user is currently in, through analysing how well the linear and logarithmic models fit to their data, the app can provide a realistic timeframe for when they might hit their next milestone.

2.2 Existing Applications

The market for weightlifting tracker apps is huge, on the Play Store alone there are tens of thousands, ranging from basic workout logging to fleshed out social apps. To fairly represent what a large portion of lifters would experience when using a lifting tracker app, on the Play Store the term “Gym Tracker” was searched, and the top 3 free weightlifting tracker applications as of 2025 were selected for analysis. These apps have a combined number of downloads of 11m+ (on Android alone).

The following section are an evaluation of these apps based on how well they fit this project’s aims.

2.2.1 Hevy

Hevy is an Android/IOS application that focuses on weightlifting tracking and analysis with social media-like features through a sleek and modern user interface.

It provides unlimited workout tracking and comes with text tutorials and custom animations with every exercise to guide inexperienced lifters. There is an entire tab dedicated to a social media-like feed, where you can like, comment and share posts, as well as follow other users. Hevy provides comprehensive workout analytics, with features like personal records and progress over time tracking.

However, Hevy has several drawbacks. For one, the free tier has many arbitrary limits to push you towards the paid tier: only four routines, seven custom exercises, and limited analysis features – such as 3 months of progress at most. The social features are quite cluttering and distracting. In terms of “Invisible Coach” features, the “How To” for each exercise is adequate if not a little useless due to being text and animation based, and automatic weight progression is not provided, relying instead on the user looking at their last performance and figuring it out themselves during a workout. There is also no ability to set your own goals, in terms of bodyweight, measurement or lifts.

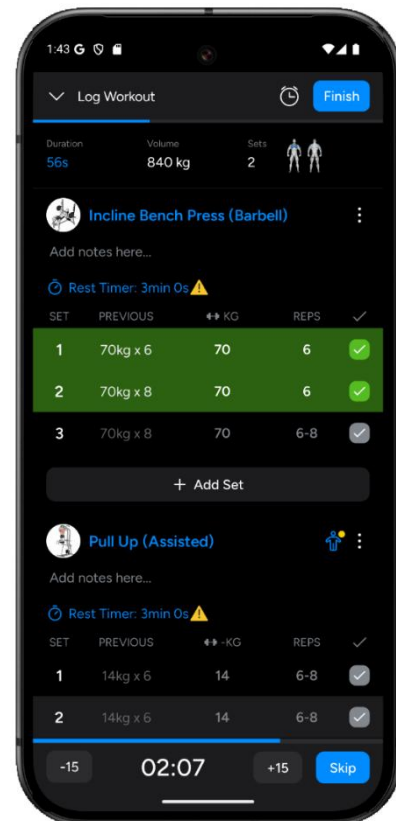


Figure 1 - Screenshot of Hevy

Overall, Hevy is a great workout tracker in and of itself, however there is a low amount of guidance and “Invisible Coach” features, and certain things are unnecessarily gated behind a paywall.

2.2.2 JEFIT

JEFIT is a veteran Android/iOS application that sells itself as an all-in-one ecosystem for lifters that prioritise content variety and analytics.

It offers a large database of 1400+ exercises, with text and video tutorials. It's "Invisible Coach" features include smart progressive overload that suggest weight and rep increases a muscle recovery map that predicts when muscle groups are ready for another session. Furthermore, there is the ability to track body measurements and weight goals beyond just your lifting stats.

JEFIT's greatest strength is also a drawback, though. It is very cluttered, with a busy and dated UI that wouldn't appeal to non-lifters, with menus and charts buried under many layers. There are also "smart" suggestions and large social workout sections that clutter the app further. The automatic progressive overload is also lacking, as it'll often recommend odd weights with decimals like "11.02kg" that are impossible to find in actual gyms.

JEFIT is a time-tested workout tracker, however it has a dated legacy application feel and has become a little bloated and unfocused.

2.2.3 Strong

Strong is an Android/iOS weightlifting tracker that features a minimalist design and aims to act as a direct replacement for a pen and paper log.

It provides workout tracking with exercise tutorials, written and animated. There is tracking for records, your weight, calories, measurements, all user inputted. And there is the ability to create workout routines as well as a few premade ones. Strong isn't cluttered by large amounts of social features, allowing the user to focus on themselves and not get distracted

Strong does auto-populate data from the previous workout but does not calculate or suggest new weights for the current session. Charts and analytics are completely inaccessible in the free version, requiring a "Pro" subscription to view progress over time, despite being a simple feature that wouldn't require much computation. Strong also lacks algorithms to forecast future strength or estimate goal completion dates.

Overall, out of the 3 applications being reviewed, Strong is closest to the goal application, due to the simple intuitive UI and lack of distracting social feeds, however it lacks any of the "Invisible Coach" features that would fulfil the aim of this project.

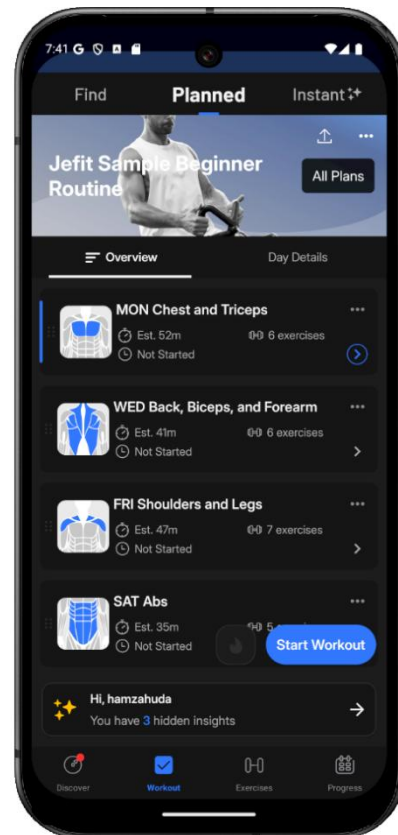


Figure 2 – Screenshot of JEFIT

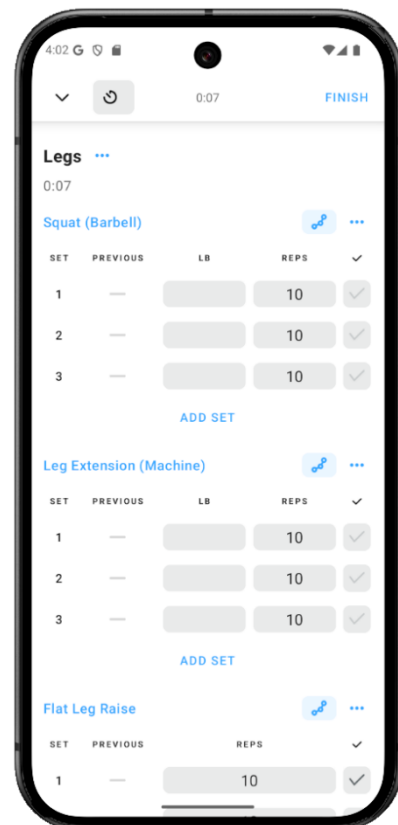


Figure 3 - Screenshot of Strong

2.3 Summary

By analysing the most popular workout trackers, I have determined that there is a market for a workout tracker with automatic progressive overload plus automatic goal weight timelines. Though other apps on the market provide subsets of Lifting Log's intended features, the reviewed applications either don't implement some features, don't bring them all together in one package or implement them poorly, as discussed previously. On top of this, most of the reviewed applications have clunky outdated UIs, plus a litany of unnecessary features that dilute the main purpose of the apps. Based on all this, we can determine what features are essential to the final product and build from there.

Chapter 3: Requirements Analysis

This chapter goes over the requirements for the final application. The functional requirements cover the key aspects of the app that are needed to fulfil the aims of the project, the non-functional are requirements that would improve the app in related and useful but not strictly necessary ways.

3.1 Functional Requirements

1. Every user should be able to sign in (via google) to their own account to access the app and their data
2. The ability to create, edit and delete workout routines
3. The ability to start a workout based on a workout routine
4. Ability to fill out a workout with your weight and reps
5. Live workout modification, changing exercises, adding sets etc. during a workout
6. Exercise list management, ship with a list of default exercises to choose from with the ability to create your own custom exercises (that get saved to your account)
7. The app should allow you to view your past performance of the current exercise during a workout
8. The app should show your performance over time of an exercise on a graph, (1rm normalised)
9. The app should automatically increment/suggest a weight and/or reps during a workout based on your past performance

3.2 Non-Functional Requirements

1. The app should be snappy and responsive to user inputs without a delay that would interrupt the flow of a workout
2. The app should allow users to track alternative activities in the gym that don't involve weights, sets and reps, (cardio, stretching, timed exercises, etc.)
3. The app should have a simple user interface that appeals to beginners and non-lifters
4. All data should be stored on the cloud and account based so that users can access their workout data from any device by signing in

Chapter 4: Design

4.1 Use Cases

Register: A user signs up for the application to create an account

Log In: A user logs into their existing account

Delete Account: A user deletes their account along with all their data

Manage Workout Templates: A user creates, edits or deletes their workout templates

Manage Workouts: A user creates, edits or deletes a workout

Manage Custom Exercises: A user creates, edits or deletes a custom exercise that is saved to their account, an exercise that isn't included in the default list of exercises

View Exercise History: A user views all their history for one exercise

View Estimated Goal Achievement Date: A user views the predicted date they will achieve their goal 1RM

Choose Regression Type: A user picks a regression type, between linear and logarithmic, to predict their goal – one of the regression types will be the recommended one

View Regression Line: A user views the regression they chose

Set Goal 1RM: A user sets a goal 1RM weight for an exercise

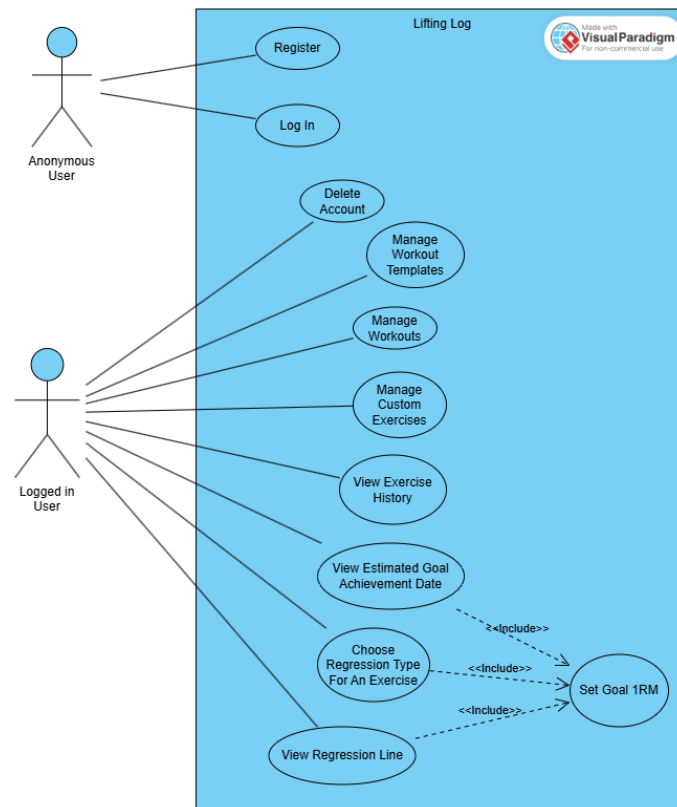


Figure 4 - Lifting Log Use Case Diagram

4.2 Development Environment and Technologies

The development of this application was split between two distinct parts, the mobile client (frontend) and the REST API server plus database (backend), both requiring different tools and frameworks – apart from the entire project being programmed with Visual Studio Code, a lightweight and customisable IDE (Integrated Development Environment).

4.2.1 Mobile Client (Frontend)

The mobile Android application was developed using React Native with the Expo framework.

Expo simplifies development by managing the native Android/iOS code and providing real-time testing and debugging of the application during development with Expo Go and Expo Development Builds – which during development were run locally on an Android emulator through Android Studio.

React Native is an open-source UI framework that is written in JavaScript/TypeScript - for this application TypeScript was the primary programming language used since it significantly reduces runtime errors during development with its strong typing. Styling was done with NativeWind (Tailwind for React Native), a styling library that allows rapid development of modern application screens through utility classes, and React Native Reusables were used as a foundation for several of the UI components which also sped up development.

A small local SQLite database was also implemented in the mobile client for fast search queries.

4.2.2 Server (Backend)

The backend RESTful API was developed using the Django REST Framework (DRF) with Python. Django provides out of the box solutions for many common issues, such as an Object Relational Mapper for database manipulation without writing SQL and a built-in admin panel. DRF provides a comprehensive set of tools to turn the Django backend into a RESTful API that communicates easily with the React Native Android application.

User management was externalised to Supabase, a cloud-based authentication provider, that securely handles sign-ups and issues JWTs for API communication that the backend server verifies.

A MySQL server was chosen for the database server and was run locally in a docker container during development.

4.3 System Architecture

As shown in *Figure 5*, the entire system making up the Lifting Log application is split amongst four main parts.

The React Native android application presents the user interface, which interacts with the backend services primarily through the API service layer. For login, sign up and the issuing of JWTs for a user, the app communicates with Supabase. Once JWT is issued, it is used in the header of all the API requests to the DRF server's endpoints. The user's token is verified using Supabase's JWKS endpoint, at which point, through the Django ORM, the primary MySQL database is manipulated as needed and communicated back to the user's application.

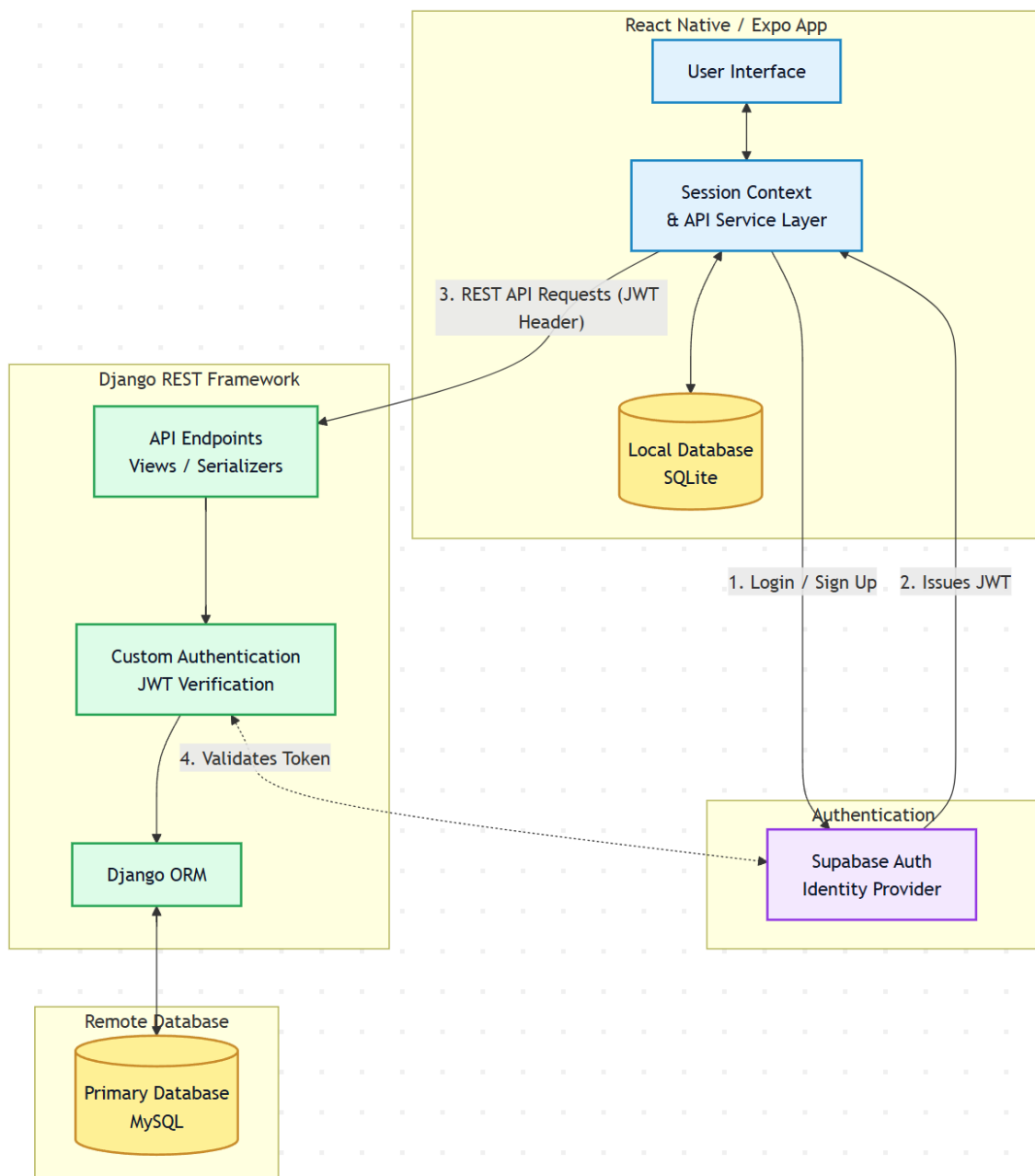


Figure 5 - Lifting Log System Architecture Diagram

4.4 Database

Figure 6 depicts the database schema of the application, implemented using Django's Object-Relational Mapping (ORM). The custom user model extends Django's default to include a `supabase_id` for synchronization with the external Supabase Authentication Service. The rest of the schema is split between two main branches, one that contains all the models for a workout template, and one that contains all the models for a workout that would be created from a template. Other models linked to the user include models for custom exercises and exercise goals. A few models included with Django have been left out for clarity.

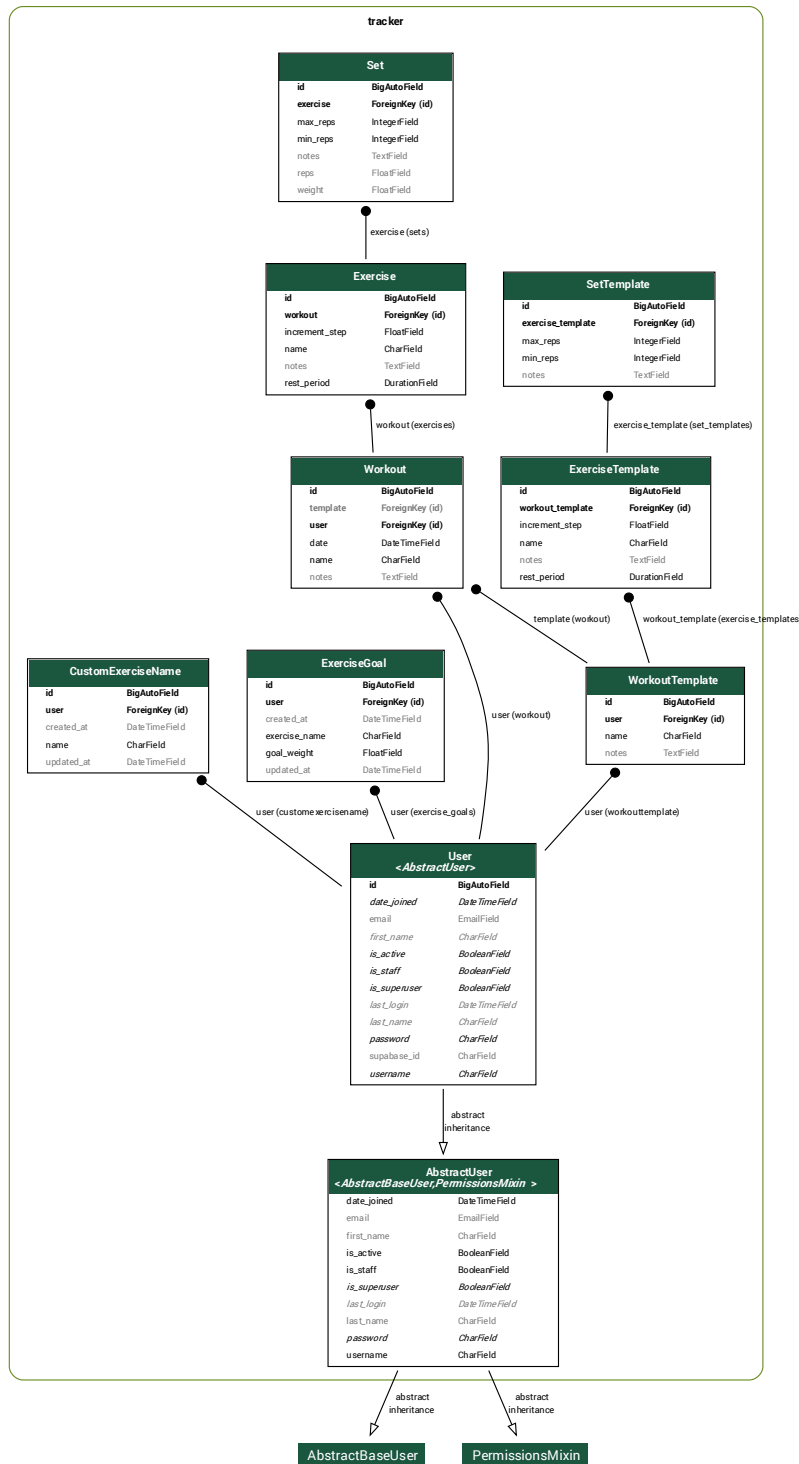


Figure 6 - Lifting Log Database Schema

4.5 Core Algorithms

4.5.1 1RM Calculation

For normalising a user's sets and reps for data analysis, the Brzycki formula (*Figure 7*) (*Brzycki, 1993*) was selected for its slightly more conservative bias and higher accuracy at lower reps.

$$1RM = W * \frac{36}{(37 - r)}$$

Figure 7 - Brzycki 1RM Formula, W = weight, r = reps

4.5.2 Strength Prediction

There are two models for predicting a future strength progression. Whichever model has a higher coefficient of determination (how well the model fits once the best fitting regression line is found), will be applied to the user's workout history.

The linear model is a straight-line relationship ($y = a + bx$), representing a constant rate of progression. This model restricts its prediction window to a maximum of one month into the future. This constraint is necessary because linear progression cannot continue indefinitely, without a cap this model would predict biologically impossible strength levels.

The logarithmic model ($y = a + b \ln(x)$) represents a curve with a constantly decreasing rate of growth, which more accurately represents a human's strength gains over time. Because this curve provides a realistic tapering effect, the logarithmic model can safely be used for longer term strength predictions.

4.5.3 Automatic Progressive Overload

Lifting Log employs automatic progressive overload through "Double Progression".

Double progression is a training method where the user works within a specific rep range for multiple sets with the same weight, for example the 6-8 rep range. Once the user successfully hits the upper bound of the rep range with the same weight for all sets, the weight is incrementally increased automatically by the application during their next session.

Since it is impossible to know what equipment every individual user has available, during template creation and/or editing, the user can choose the amount they want the app to automatically increment their weight by for each exercise.

Chapter 5: Backend Implementation

5.1 Workspace Setup

Before developing the backend, a MySQL database that would be running locally on the same computer as the Django server and Android emulator during development was set up. To streamline development the MySQL server was deployed inside a Docker container (Figure 8). Running the database in a docker container allows for future portability, keeps the host system clean and simplifies the configuration process.

During development a tasks.json file was created to prevent repeatedly having to start the Django server and Expo server every time I wanted to work on the project (Figure 9). This is a VS Code feature that allows you to define custom tasks that run when the project is opened.

Git was used throughout the entirety of the project, and commits were pushed consistently throughout to GitHub to prevent data loss. Git provided a safety blanket whenever there were bugs or while new features were being developed. Certain files like ".env" files containing sensitive environment variables were put in a ".gitignore" file.

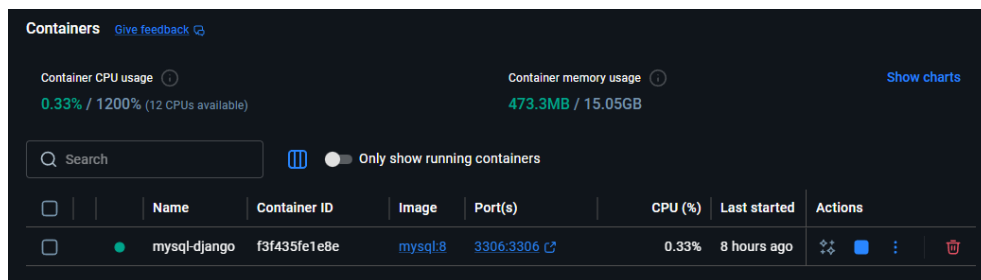


Figure 8 - MySQL Docker container

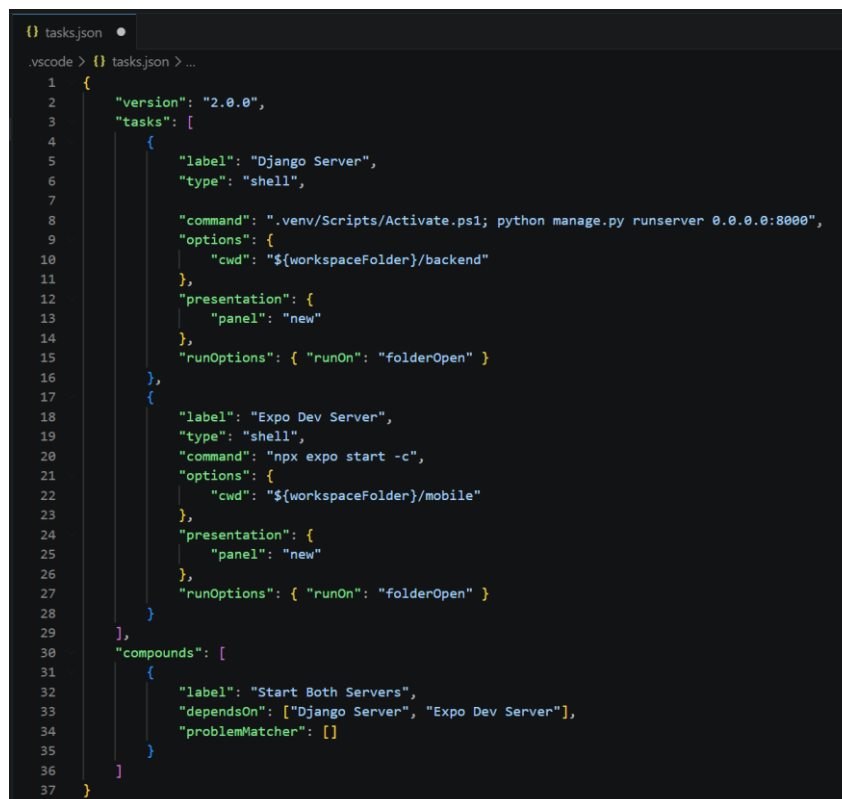


Figure 9 - tasks.json file

5.2 Models

The models were implemented with all the necessary fields to store training and user data; however, they also contain methods and class methods involving data manipulation.

Below we'll go through how some of the models and their fields were defined, then we'll look at a few of the model methods.

5.2.1 Model Fields

```
class Workout(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    date = models.DateTimeField()
    notes = models.TextField(blank=True)
    template = models.ForeignKey(
        "WorkoutTemplate", on_delete=models.SET_NULL, null=True, blank=True
    )
```

The above code shows how the Workout model's data was defined, as shown, every workout has a user field, where `"on_delete=models.CASCADE"` defines that when a user is deleted, this and other dependent models should be deleted as well.

The name, date and notes fields are self-explanatory, with their corresponding data types that match MySQL data types. The last field, template, is a foreign key to another model "WorkoutTemplate", that defines which workout template this workout was created from. In this case, when the workout template is deleted, this field is set to null instead of the workout being deleted, which is the preferred behaviour as you wouldn't want your workout history to be lost when you delete a workout template.

```
class User(AbstractUser):
    supabase_id = models.CharField(max_length=255, unique=True, null=True,
blank=True)
    email = models.EmailField(("email address"), unique=True, null=True,
blank=True)
```

This code snippet is of the User model, which subclasses the default Django User model and adds two fields, most important of which is the `"supabase_id"` field, since Supabase is being used as an external auth provider for this application.

```
class Exercise(models.Model):
    workout = models.ForeignKey(
        "Workout", related_name="exercises", on_delete=models.CASCADE
    )
    name = models.CharField(max_length=100)
    rest_period = models.DurationField()
    notes = models.TextField(blank=True)
    increment_step = models.FloatField(default=2.5)
```

This model definition has a foreign key field with a `"related_name"` argument, which defines how we would refer to this model if accessing it from the Workout model. The

last field is “*increment_step*”, which will be used when a user hits the top rep range of all their sets and their weight is automatically incremented for them.

5.2.2 Model Methods

```
@classmethod
@transaction.atomic
def create_from_template(cls, user, template, date):
```

This method is defined within the Workout model and is key to how the application functions. The “*@classmethod*” decorator binds this method to the class rather than its instances, which lets this method function as a factory for creating workouts. The “*@transaction.atomic*” decorator is present as this method manipulates the database, it bunches up all the database operations done during the method into one transaction, ensuring data consistency and integrity in case of an exception.

The rest of this method is quite repetitive as it loops through a workout template’s structure to copy it into a new workout, but this part is unique:

```
if previous_exercise:
    previous_sets = previous_exercise.sets.all()
    if previous_sets.exists():
        first_weight = previous_sets.first().weight

        if first_weight is not None:
            all_same_weight = all(
                s.weight == first_weight for s in previous_sets
            )
            all_hit_top_reps = all(
                s.reps is not None and s.reps >= s.max_reps
                for s in previous_sets
            )

            if all_same_weight and all_hit_top_reps:
                autofilled_weight = (
                    first_weight + exercise_template.increment_step
                )
            else:
                autofilled_weight = first_weight
```

This logic looks at the last time this exercise we are adding to our new workout was performed, it looks through all the sets that were done. If the top of the rep range was hit for the same weight for all sets, it auto fills the weight when creating this workout, by incrementing it by the “*increment_step*” field present on the workout template model. This essentially is the automatic progressive overload feature.

Here is one more model method:

```
def delete(self, *args, **kwargs):
    """
    Overrides the default delete method to also delete the user from Supabase.
    """
    if self.supabase_id:
        try:
            supabaseAdmin = create_client(
                settings.SUPABASE_URL, settings.SUPABASE_SECRET_KEY
            )

            supabaseAdmin.auth.admin.delete_user(self.supabase_id)

            logger.info(
                f"Successfully deleted user {self.supabase_id} from Supabase."
            )
        except Exception as e:
            logger.error(
                f"Failed to delete user {self.supabase_id} from Supabase: {e}"
            )

    super().delete(*args, **kwargs)
```

This method is present on the “User” model. It does as the docstring describes, using the Supabase python library, it deletes a user from the external auth database, which it accesses with the “SUPABASE_SECRET_KEY” stored safely on the backend Django server.

5.3 Custom Permissions

```
class IsObjectOwner(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        return obj.user == request.user
```

Above is one of the custom permission classes, created by subclassing “*permissions.BasePermission*”. This is used in the DRF views, acting before the view processes the data but after user authentication, it gatekeeps a user from acting if they are not the owner of the object they are trying to manipulate. Will cause a 403 Forbidden HTTP status, securing users personal workout data.

5.4 Views and API Endpoints

The API endpoints that the mobile app interact with were handled using DRF’s (Django Rest Framework’s) ViewSets. A ViewSet is a class-based view that comes with lots of basic functionality built in already (Create, Read, Update, and Delete operations), routing them to their correct HTTP methods.

5.4.1 QuerySet Filtering

```
class WorkoutViewSet(viewsets.ModelViewSet):
    serializer_class = WorkoutSerializer
    permission_classes = [IsAuthenticated, IsObjectOwner]

    def get_queryset(self):
        user = self.request.user
        return Workout.objects.filter(user=user).order_by("-date")
```

The above code demonstrates how, at the database level, data privacy and user separation is enforced. By overriding the “*get_queryset*” method in the “*WorkoutViewSet*”, the API ensures that whenever a user requests any operation involving workouts, only the workout objects linked to the user making the request (“*self.request.user*”) can be affected. Notice the workout objects are also ordered by their date at the database level, which saves resources as databases are more optimised for these types of operations over doing this sorting in the mobile app.

5.4.2 Custom API Actions

While ViewSets handle standard CRUD operations, certain “Invisible Coach” features required custom endpoints, here is one:

```
@action(detail=False, methods=["get"], url_path="last-performance")
def last_performance(self, request):
    exercise_name = request.query_params.get("name")
    current_workout_id = request.query_params.get("workout_id")

    ... validation logic ...

    last_exercise = (
        Exercise.objects.filter(
            workout__user=request.user,
            name=exercise_name,
            workout__date__lt=current_workout.date,
        )
        .order_by("-workout__date")
        .first()
    )
```

The “*@action*” decorator turns this function contained within the “*ExerciseViewSet*” into a custom API route, “*/last-performance*”. This endpoint takes two query parameters, the “*exercise_name*” and “*current_workout_id*”, then returns how the user performed on that specific exercise the last time the exercise was performed, with the database doing all the heavy lifting of sorting and manipulating the data.

5.5 Serializers

In DRF, serializers convert the Django model instances (as described in 5.2) into native python datatypes that can then easily be rendered into JSON (JavaScript Object Notation). They also can reverse the process, taking incoming JSON data, validating it, then passing it along to be used in ViewSet or model methods.

5.5.1 Nested Serializers

```
class WorkoutSerializer(serializers.HyperlinkedModelSerializer):

    user = serializers.ReadOnlyField(source="user.username")
    exercises = ExerciseSerializer(many=True, required=False)

    class Meta:
        model = Workout
        fields = ("id", "url", "user", "name", "date", "notes", "exercises", "template")
```

The fields variable defines all the model fields to be passed through the serializer.

To minimise API calls and improve the mobile app's performance, nested serializers were used, by including `"exercises = ExerciseSerializer(many=True)"` a single get request for workout returns all the associated exercises, and since a similar thing is employed in the `"ExerciseSerializer"`, all the sets are fetched too. The result is a single cohesive JSON payload received by the frontend that contains the entire workout and all its nested objects in one API call.

5.5.2 Connecting Custom Methods

```
def create(self, validated_data):
    user = self.context["request"].user
    return WorkoutTemplate.create_with_exercises(
        user=user, template_data=validated_data
    )
```

This create function overrides the default one in the `"WorkoutTemplateSerializer"`, instead, the custom class model method `"create_with_exercises"` is called with the validated data from the serializer passed to it.

```
def create(self, validated_data):
    user = self.context["request"].user
    template = validated_data.get("template")
    date = validated_data.get("date", timezone.now())

    if template.user != user:
        raise serializers.ValidationError(
            "You do not have permission to use this template."
        )

    return Workout.create_from_template(user=user, template=template,
date=date)
```

The same thing is done here in the `"WorkoutSerializer"`, though additionally the owner of the template is validated before the custom class method is called to create a workout from a workout template.

5.6 Authentication

Since user authentication is externalised to Supabase instead of passwords being managed locally, the Django backend only needs to verify the JWTs Supabase issues.

5.6.1 JWT Verification

```
jwks_url = f"{settings.SUPABASE_URL}/auth/v1/.well-known/jwks.json"
jwks_client = PyJWKClient(uri=jwks_url, cache_jwk_set=True, lifespan=600)
signing_key = jwks_client.get_signing_key_from_jwt(token)

payload = jwt.decode(
    token,
    signing_key.key,
    algorithms=["ES256"],
    audience=settings.SUPABASE_AUDIENCE,
)
```

The above is the core logic from the custom authentication class “*SupabaseAuthentication*” that subclasses DRF’s “*BaseAuthentication*”. Supabase’s JWKS (JSON Web Key Set) is fetched and used to verify the token provided by the mobile application when an API call is made.

5.6.2 Local User Synchronisation

Once the token is verified, the “*supabase_id*” is extracted from the token and used as follows:

```
user, created = User.objects.get_or_create(
    supabase_id=supabase_id,
    defaults={
        "email": email,
        "username": email,
    },
)
```

The “*get_or_create*” method either creates a local Django user on the MySQL database, or it gets the user with the corresponding details. A local account is needed since the MySQL database needs a user to correspond with workouts, workout templates, custom exercises, etc. but notice how there is no password field as Supabase handles all the authentication.

Chapter 6: Frontend Implementation

The mobile client serves as the user-facing half of the Lifting-Log application, it was developed after the backend. This Expo project uses Expo router, a file-based router where the app's routes are automatically defined based on the file structure of the project, it supports stacked screens, tabs, and dynamic routes without any manual configuration.

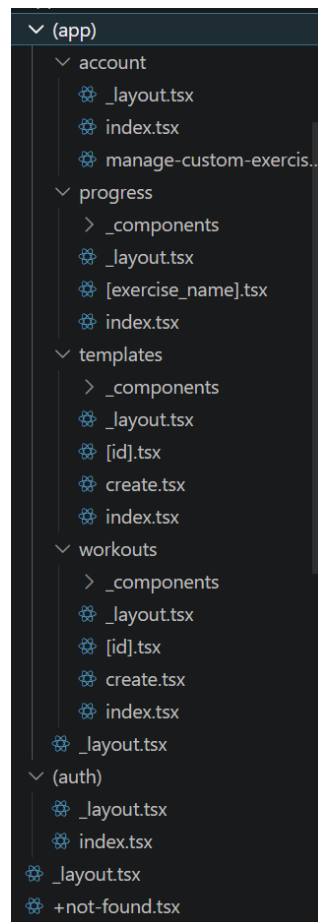


Figure 10 - Expo Router based file layout

6.1 User Authentication and Management

For streamlined and frictionless user onboarding, the mobile application only accepts sign up and login with Google OAuth (an open standard for allowing users to grant applications access to their Google account information, without handing over their password). This is handled securely through Supabase.

6.1.1 Supabase Client Initialisation

The Supabase docs provided a step-by-step guide on how to set up Google sign in with an Expo React Native application which streamlined the development process.

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {  
  auth: {  
    storage: AsyncStorage,  
    autoRefreshToken: true,  
    persistSession: true,  
    detectSessionInUrl: true,  
  },  
});
```

The above code initialises the Supabase client and uses “*AsyncStorage*” to ensure users remain logged in even if the application is closed, and the tokens are automatically refreshed (“*autoRefreshToken: true*”).

6.1.2 Auth State Management

Using React Context (a way to make data available throughout the whole application), the various screens of the app know the user’s authentication status which prevents them from accessing protected routes.

The “*SessionProvider*” wraps the whole application, listening for any changes in the authentication state. It exposes the session and whether the session is loading throughout the entire app through a “*useSession*” hook.

6.1.3 Sign In Screen

The sign in screen is comprised mainly of the “*SignInForm*” component, with the login function being inside the “*SocialConnections*” component, the “*loginWithGoogle()*” function was written using the Supabase documentation.

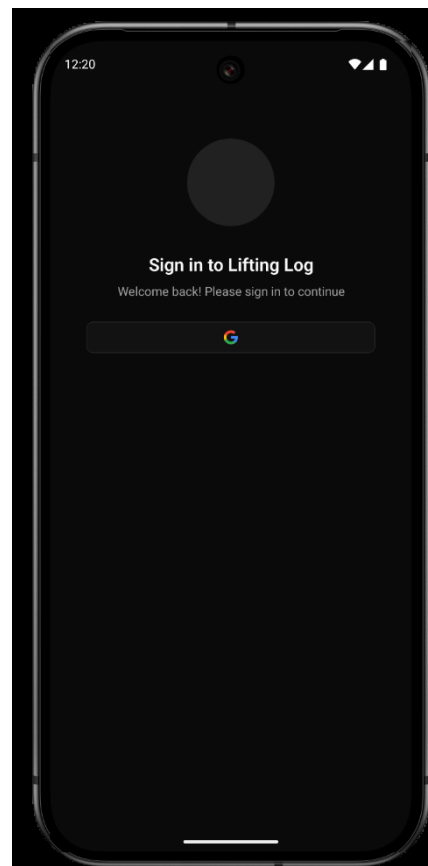


Figure 11 - Login Screen

6.2 UI Architecture

To ensure a visually consistent, maintainable, and scalable user interface, the application uses NativeWind and React Native Reusables.

6.2.1 NativeWind

NativeWind is Tailwind CSS for React Native projects, with its utility classes and inline styling. It was used in this project to allow quick and consistent styling when screens were being built. The “*theme.ts*” and “*tailwind.config.js*” files map colours to variables that are used throughout the project, like “*bg-background*” or “*text-muted-foreground*”. Below is an example of the classes and variables being used for inline styling:

```
<View className="flex-row py-1 mb-2 items-center">
  <View className="flex-1 flex-row items-center">
    <Text className="w-12 text-foreground text-lg text-center">
```

6.2.2 React Native Reusables

RNR is an open-source UI component library of premade, highly customisable UI components, built with NativeWind. Using RNR components I was able to skip typing out common boilerplate for popular components or building my own component library, instead RNR provided these to allow the rapid development of Lifting Log’s screens, only requiring me to customise the components when it was needed.

6.3 API Service Layer

The API service layer in “*api.ts*” is where all communications with the backend Django server pass through. It is built using Axios, a promise-based HTTP client that provides many benefits over the native Fetch API- such as better syntax, built-in error handling and interceptors.

6.3.1 Request Interceptor

Since the JWTs are auto refreshed by Supabase, all that must be done by the request interceptor is that the active session’s “*access_token*” must be attached to every outgoing API call.

```
api.interceptors.request.use(
  async (config: InternalAxiosRequestConfig) => {
    const {
      data: { session },
    } = await supabase.auth.getSession();
    if (session) {
      config.headers.Authorization = `Bearer ${session.access_token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  },
);
```

6.3.2 API Functions

The rest of the “*api.ts*” file is made up of exported functions for every API call done throughout the rest of the application. For example, the function below “*fetchTemplate*” asks for an “*id*” and will return a “*WorkoutTemplate*” inside a response object.

```
export const fetchTemplate = async (id: number | string) => {  
  const response = await api.get<WorkoutTemplate>(  
    `/workout-templates/${id}/`,  
  );  
  return response;  
};
```

6.4 Local Database for Custom Exercises

To allow instant search queries for custom exercises, even though a user’s custom exercises need to be saved to their account on the backend, a small local SQLite database was implemented that stores the exercise dictionary locally. The local database contains a list of default exercises that come with the application as well as the user’s custom exercises. This database is synced with the backend every time the database is initialised, where new custom exercises are flagged on creation to be synced or deleted.

When searching, the SQL queries filter the exercise names instantly due to it all being locally done, significantly reducing the load on the backend API by minimising constant API calls, providing a snappy user experience.

Below is a code snippet of the return value of the “*searchExercises*” function, this is the SQL executed as every letter of the search query is typed to filter the results.

```
return (  
  await db.getAllAsync<{ name: string }>(  
    "SELECT name FROM exercise_names WHERE name MATCH ? AND  
is_deleted = 0 ORDER BY rank",  
    [ftsQuery],  
  )  
).map((row) => row.name);
```

6.5 Templates

This tab is where the user manages their workout templates. The main screen retrieves a list of all the user's saved workout templates, an options button on each template causes a modal to pop up that gives the option to duplicate or delete that template.

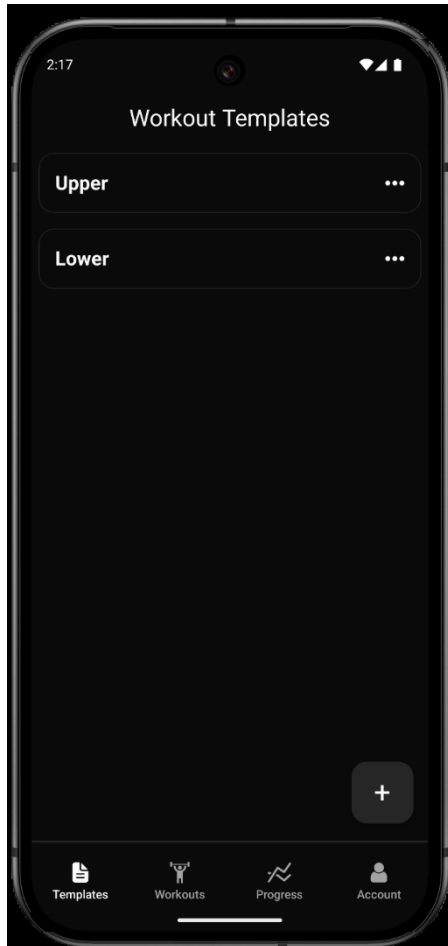


Figure 13 - Template List Screen

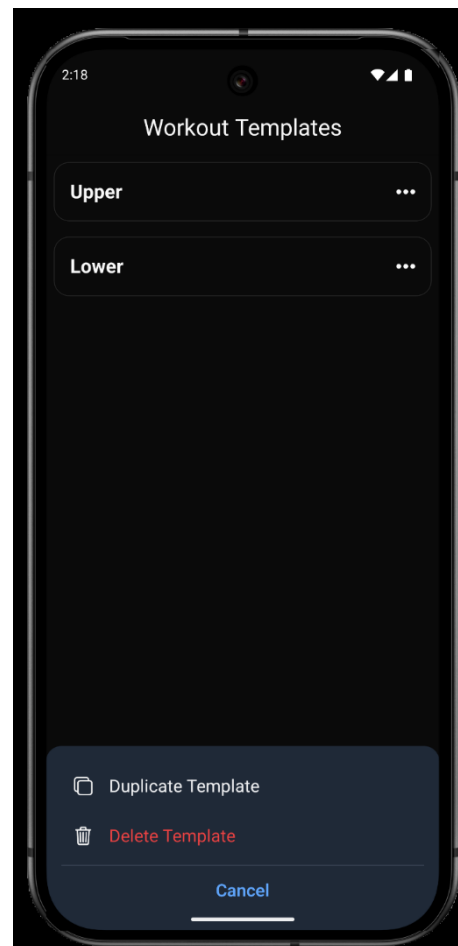


Figure 12 - Template Modal

There is a “*FloatingActionButton*” in the bottom right of the main template page that pushes the user to the template creation page when pressed. When a template is pressed, the user is taken to the template editing page. Both the template editing and template creation page both use the “*TemplateForm*” component but have the resulting form data handled differently. In the template form, when an exercise is added and the name is tapped to be inputted, a modal pops up that allows the user to search through the default exercises plus their custom exercises.

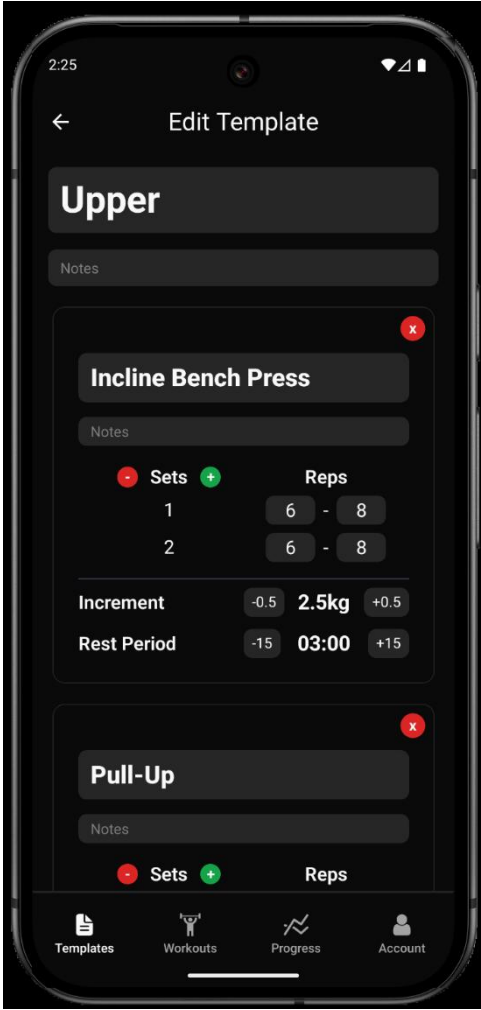


Figure 15 - Edit Template Screen (Template Form)

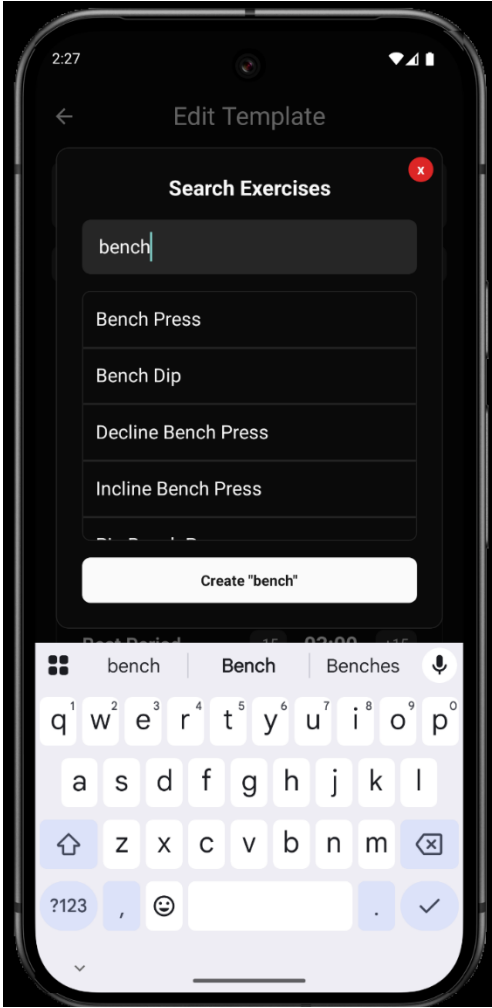


Figure 14 - Search Exercises Modal

6.6 Workouts

The workouts tab immediately shows you a list of all your workouts, sorted from latest to oldest, with the number of the workout on the left, workout name, and date. The options button gives you the option to delete the workout or edit the date or time.



Figure 17 - Workouts Screen

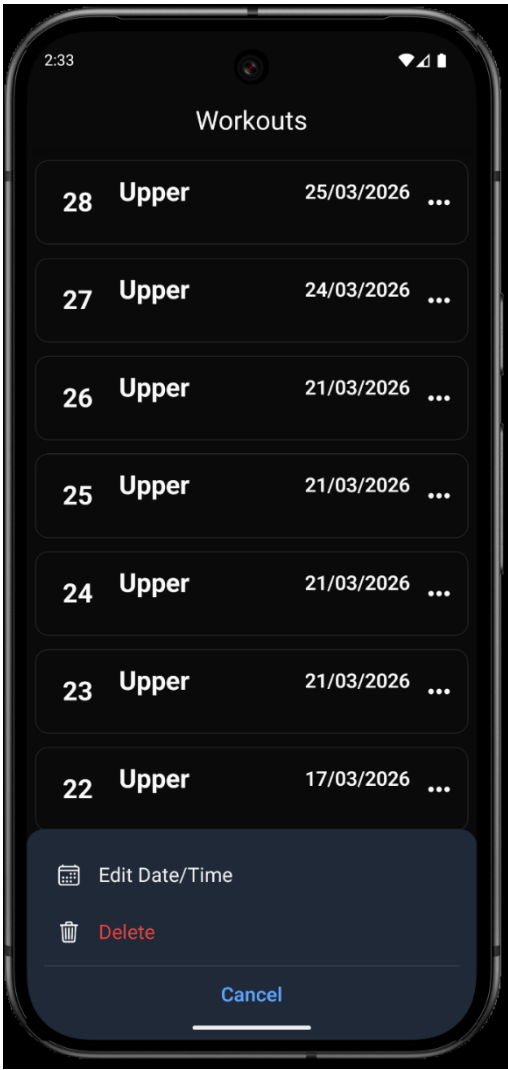


Figure 16 - Workout Modal

The “*FloatingActionButton*” takes you to the workout creation page, where you can create a workout from a template, clicking a workout takes you to the that workout’s screen, where the user will be during a workout, where they can fill out their weights and reps, start/stop or close a rest timer, write down any notes. The pencil icon in the top right of the workout page enables editing mode, where you can add or remove sets.

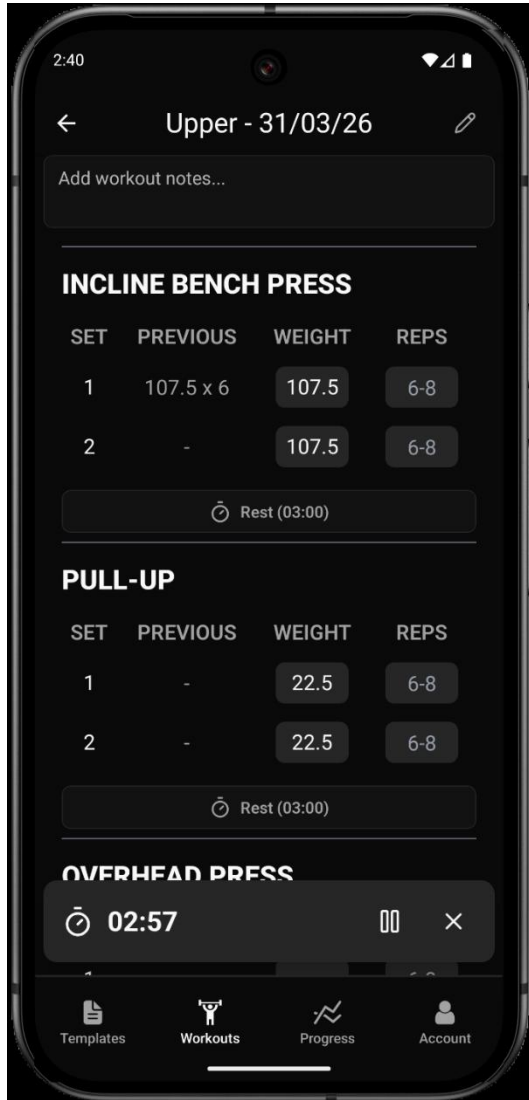


Figure 18 - Workout Screen

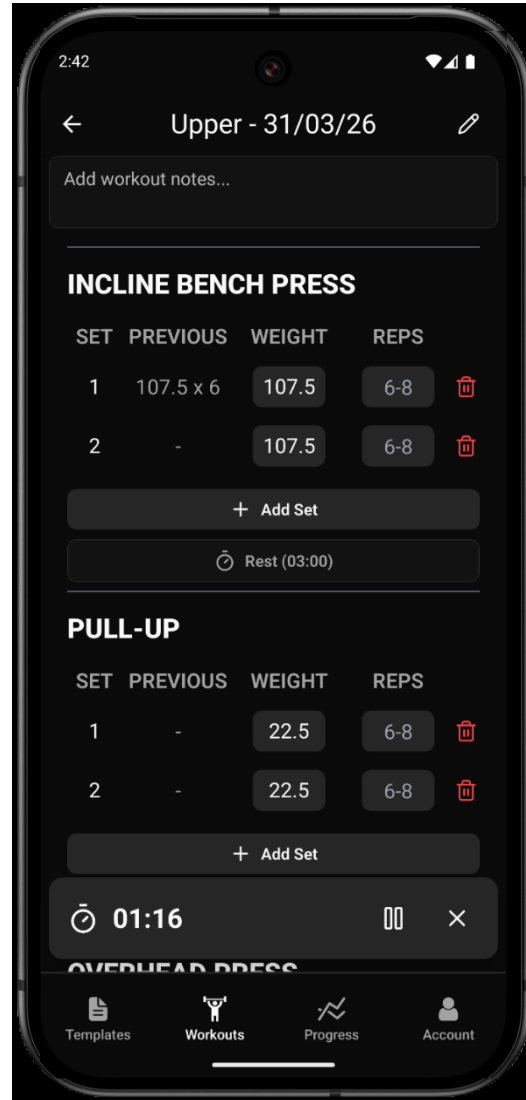


Figure 19 - Edit Workout Enabled

6.7 Progress

The progress tab’s home screen shows a list of all the exercises you’ve done, sorted from newest to oldest, when an exercise is pressed, you are taken to that exercise’s progress screen. The progress screen shows a graph of your performance in that exercise over time (normalised to the highest 1RM during that workout), and at the bottom of the screen is a list of all the times the user performed that exercise.

Right beneath the graph, is the goal setting field. Once the user enters a goal weight, their historical workout data is processed using both a linear regression model ($y = a + bx$) and a logarithmic regression model ($y = a + b \cdot \ln(x)$). The implementation of the regression lines in “*prediction.ts*” is handled by the “*regression.js*” library.

The application then calculates the coefficient of determination for both models to decide which one fits the data better, and the optimal model is indicated to the user by a star icon next to the regression type. The line of best fit is then applied to the chart for the user to see, and an “Estimated Time Remaining” text appears underneath based on a prediction of the chosen regression model, with the cap set to 30 days for linear and 1 year for logarithmic as linear models are inaccurate for long term goal setting due to the naturally logarithmic nature of strength gains.



Figure 20 - Progress Screen

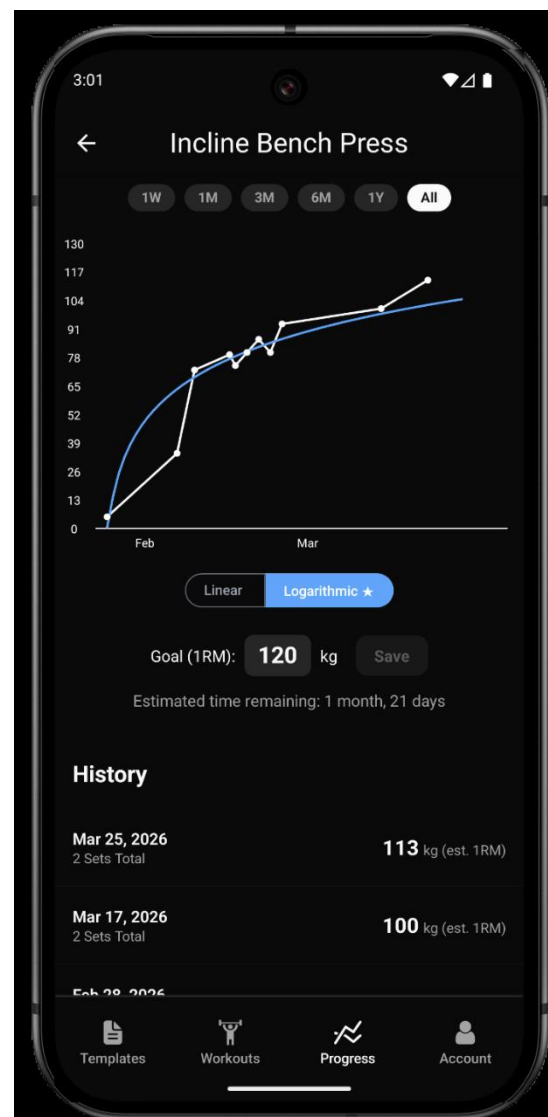


Figure 21 - Progress Screen with Goal Set

6.8 Account Tab

In terms of the user's account, the account tab lets the user view the current account they are signed in on, sign out, or delete their account. The manage custom exercises button takes you to a page where you can see all the custom exercises you have created that weren't a part of the default exercises that come with the app. It allows searching, creating, editing, and deletion.

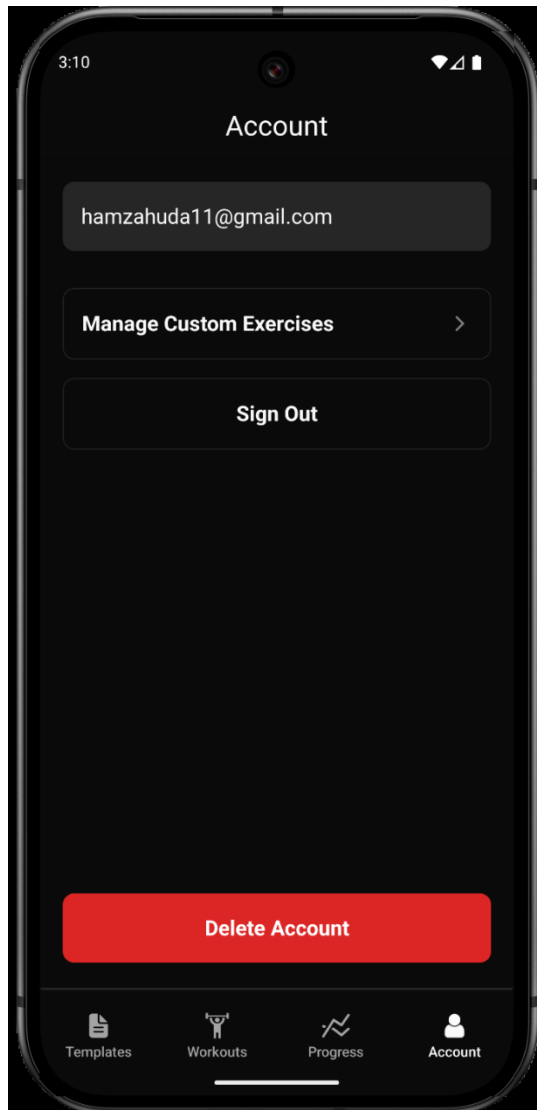


Figure 22 - Account Screen

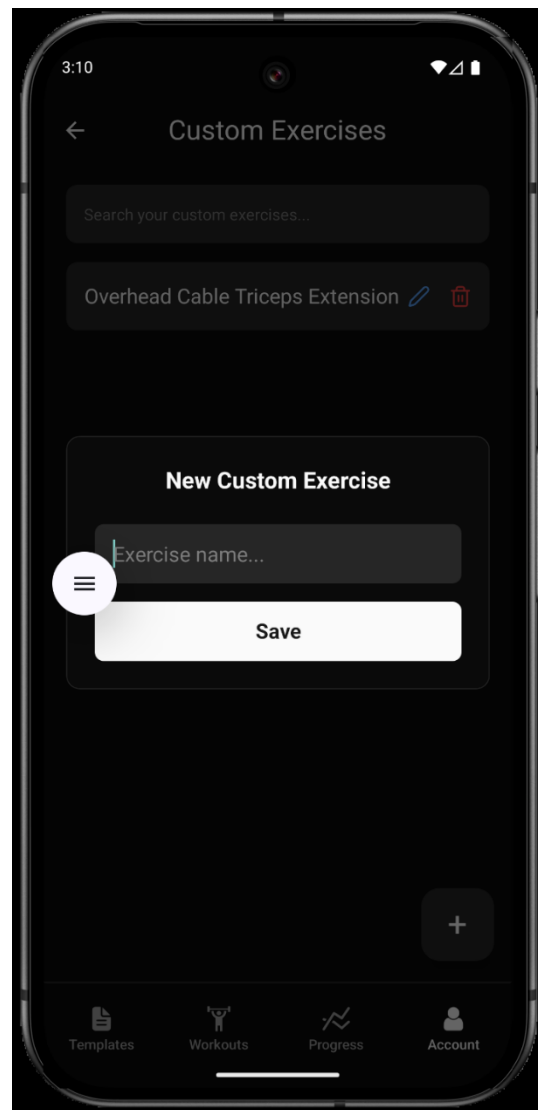


Figure 23 - Manage Custom Exercises Screen, Adding an Exercise

Chapter 7: Testing

Testing of this application was split into two phases, automated backend testing to verify business logic and data integrity, and manual/user testing to evaluate the React Native application's user experience. The goal of these tests is to ensure robustness, security and usability of the application.

7.1 Automated Backend Tests

The backend tests were built with Django Rest Framework's *"APITestCase"*. These tests were split amongst three files, one for model logic, one for API functionality, and one for permissions.

7.1.1 Model Tests

Unit tests were written to verify the fundamental business logic. For example, the automatic progressive overload on creation of a workout was rigorously tested. One test simulates a user logging a workout where they hit their maximum rep range, and asserts that when a new workout is created, the weights are automatically filled in and incremented by the template-specified increment step. While another test simulates the user not hitting the top end of the rep range and asserts that the newly created workout auto fills with a maintained not incremented weight.

```
for s in new_exercise.sets.all():
    self.assertEqual(
        s.weight, 100 +
        self.template.exercise_templates.first().increment_step
    )
```

Figure 24 – Progressive overload unit test assertion

7.1.2 API and Integration Tests

The integration tests were designed to test if the REST API endpoints handled requests correctly. One test, for example, tests the *"single-exercise-history"* endpoint, which is supposed to return the entire exercise history for an exercise, oldest to newest. Instead of testing a method directly, an API call is simulated and the response is tested.

```
url = reverse("exercise-single-exercise-history")
response = self.client.get(url, {"exercise_name": "Deadlift"})
self.assertEqual(response.status_code, status.HTTP_200_OK)
self.assertEqual(len(response.data), 2)
self.assertEqual(response.data[0]["sets"][0]["weight"], 140)
self.assertEqual(response.data[1]["sets"][0]["weight"], 145)
```

Figure 25 – "single-exercise-history" test

7.1.3 Permissions Tests

Given how personal fitness and health data is, the custom permissions had to be tested rigorously too. For example, for the testing of the *"IsObjectOwner"* permission, an authenticated user tries to retrieve another user's workout, tries to modify their template, and then tries to check their workout list to see if another user's workout is viewable. In the first two cases a 404 error is expected, and for the last we assert that the malicious user finds no workouts.

7.1.4 Test Table

Table 7.1 shows a small selection of the implemented automated backend tests to demonstrate the thought process behind designing the tests.

Table 7.1: Selection of Backend Tests

Test Category	Component Tested	Description	Expected Outcome	Status
Model	Progressive Overload	User hits max target reps on an exercise, create a new workout from the same template.	The new workout has its weight automatically increased and prefilled by the <i>"increment_step"</i>	Pass
Model	Data Synchronisation	A user account is deleted	A call is made to the Supabase API to delete the user from the server	Pass
API	Workout Instantiation	A POST request to create a new workout from a template	A new workout identical to the template is generated and a 201 status is returned	Pass
API	Progress Retrieval	A GET request is sent for the last performance of an exercise	All relevant information from the last time the exercise was performed is returned	Pass
Permissions	Object Permissions	User B tries a GET request on a workout owned by User A	The request fails and a 404 or 403 status is returned (depending on implementation)	Pass

7.2 Manual Frontend Tests

For the testing of the frontend, due to time constraints, testing was done manually as each feature was added. The application was tested locally on the development PC with an Android emulator and on my personal Android device to ensure functionality. The goal here was also to assess the fluidity and simplicity of navigation and use, on top of verifying everything worked.

All use cases described in the use case diagram (Figure 4) were tested, but here are some core flows that were tested with a small description to show the granularity of manual testing:

- Authentication Flow: Registration of a new account on a fresh app, logging out, and logging back in all with the google account integration
- Workout Logging: Interacting with the numeric inputs for weights and reps to complete a workout
- Data Visualisation: Ensuring the time filter buttons were updating the workout history chart correctly in the progress tab

7.3 User Tests

After all the functionality testing, to ensure the aim of this application of being an “Invisible Coach” and a beginner friendly app was fulfilled, a testing phase was done with real users.

7.3.1 Methodology

A small group of 4 participants were given access to the application, half on their Android devices and half through the Android Studio emulator. They were asked to complete specific tasks without any guidance; these were the three main tasks:

1. Create a workout template for a “Pull Day” with a “Barbell Row” as one of the exercises
2. Log three workouts, filling out the weight and reps
3. Locate and view their progress for the “Barbell Row”

Other tasks included but were not limited to:

- Add a custom exercise to a workout template
- Save a workout goal
- Edit the date/time of a workout
- Delete a custom exercise you’ve created
- Edit your last workout to hit the top rep range, then create a new workout

7.3.2 Feedback

The user testing provided lots of insights as to how well the application fulfils its intended purpose:

- All users, who ranged from beginner to intermediate lifters, found the UI very easy and intuitive to navigate, with little difficulty in navigating the menus to complete their tasks.
- The interface was praised for it’s minimalist and less cluttered design without unnecessary screens
- The performance of the app was perfect from what they could tell through use
- The automatic weight calculation was reported as very helpful: *“you can turn off your brain and focus on working out with this.”*
- The progress tab and all its features were commented on positively: *“This is literally the only feature I would use, other than workout tracking, in a workout app”*

Any areas commented on during testing to improve the app were implemented, these included making the manage exercises button more prominent and letting a user click a specific workout in the progress tab and be navigated to that workout’s page for editing and viewing.

Chapter 8: Evaluation

In this chapter the final project is evaluated against the outline created at the start of the project in chapters 1-3.

8.1 Requirements

A key marker of the success of this application can be evaluated based on requirements outlined in chapter three.

In terms of functional requirements, they were all successfully met, making the application a success on a foundational level.

In terms of the non-functional requirements, though user feedback suggested performance was snappy and the interface was simple, and the cloud-based account storage was implemented successfully, the non-functional requirement of providing tracking for alternative activities (cardio, stretching, etc.) was not provided.

8.2 Resolution of Research Questions

The core research questions presented at the beginning of this project were addressed with varying levels of success.

8.2.1 How can historical lifting performance be mathematically modelled to predict future strength gains accounting for new and experienced lifters?

This was solved by first normalising the user's historical workout data with a 1RM formula, then applying either a linear or logarithmic regression to the data depending on which had a higher coefficient of determination. The linear model would therefore be applied to beginners experiencing a rapid, steady gain in strength, while the logarithmic model would fit to the diminishing returns of an experienced lifter.

8.2.2 How much does automatic progressive overload improve a user's workout?

Because the time frame of user testing was limited, positive feedback about the progressive overload feature could only be approximated by the test users, though they did all think it would benefit their training. A longer test of months per person, where they use one app where there is no automatic progressive overload and one where there is, would be required to substantiate just how much better this feature makes the user experience.

8.2.3 What features of market available weightlifting trackers are unnecessary for an average user?

Through the market research and during user testing, it was found that things like social feeds and suggested workout templates, anything that distracted from the lifting, was unnecessary for most users, though the sample size of people asked was too small to draw a definitive conclusion from.

8.2.4 How useful is strength prediction for motivating the end user to keep lifting?

By providing a target date for a strength goal, all users found this feature to be a novel and motivating tool. By providing an achievable goal that the user is predicted to achieve if they stay consistent, the predicted goal date becomes a source of extrinsic motivation.

Longer term tests over months would have to be done to determine how much this improves weightlifting consistency.

8.3 Limitations

Though the core objectives were fulfilled, there are some limitations that were met.

Although automated tests were applied to the backend, the frontend was tested manually, unfortunately, due to time constraints I couldn't learn then implement E2E (end-to-end) testing, which would have automated user flow simulations and provided a high level of confidence in the system's stability and performance.

Also, due to the nature of weightlifting, the ideal timelines of testing an application would be months or years, since real strength gains takes a long time and consistency. Unfortunately, the user evaluation was restricted to a small pool of friends and family with a short time frame, only enough to get a snapshot of user satisfaction instead of a validation of long-term effectiveness of the final app.

Only near the end of the applications development did the realisation that certain gyms, other than the ones I've visited, may lack cellular reception or good Wi-Fi occur to me. This means that since this is a cloud-first tracker, poor signal would cause lag when logging a workout as API calls are made often during logging. If there were more time I would have pivoted to an offline-first system where the data is synced to the cloud in the background once a good connection is established.

Chapter 9: Conclusion

This project pushed me to the limit of my technical abilities and forced me to learn lots of new things. This chapter goes over my personal journey, development and achievements.

9.1 Achievements

There was a steep but rewarding learning curve during the development of this application. The idea of this app had been in my mind for a long time, but I had none of the skills needed, in fact, at the beginning of the project, I only had a light grasp of Django and absolutely no idea how mobile development worked. I was forced to do lots of research into the mobile development world, understanding general landscape of the modern ways of building mobile applications, before choosing the technologies and development environment that I thought would be most suited for this project.

Once I started development it was hours of sifting through documentation, constantly learning and experimenting, even setting up the local MySQL database in a Docker container then connecting it to my Django server required me to do hours of research as this was all completely new to me. There were so many technologies I had zero understanding of that I had to learn, Expo, JWTs, DRF, Supabase, React Native... I had to learn everything from the ground up. However, once I got into the flow of the development, the progress was extremely rewarding and almost addictive, I found myself thinking of features and ways to fix bugs in my free-time or during my part-time job.

I'm extremely proud of the fact I now have a finished tangible mobile application, that if I wanted to I could deploy to the Play Store, it has always been a dream of mine to be able to bring an idea to life in this manner and I feel like I've really achieved it with this project, and proven to myself that if I put in the effort I can achieve anything I've set out to do.

9.2 Challenges

Using third party authentication with social login was one of the hardest things I did during this project, I spent many, many hours reading and researching on how to integrate Supabase with DRF and was pushed far out of my comfort zone in developing custom authentication. However, I'm very grateful I chose this path, as it taught me a lot about privacy and safety in mobile development, which is especially important when you're essentially dealing with very personal health data. DRF was also my first introduction into what a REST API was, a term I had heard often yet had no idea what it was in practice.

Learning the frontend development was also a big hurdle, I have no experience with React, so React Native was completely unfamiliar to me, it took many YouTube crash courses and rereading of the official documentation for me to wrap my head around the unfamiliar way react works, as it is unlike any technology I've worked with before. The struggle was only compounded by the fact that I originally was creating and styling lots of repeated components myself before discovering RNR's components.

9.3 Future Work

With more time available I would like to add some quality-of-life features that I missed out on, they weren't necessary to the functionality of the application, but I think they would greatly enhance the user experience.

Offline-First Architecture: As outlined in the limitations section (8.3) I would much prefer if all the workout data was stored immediately on device, then later synced to the cloud, though this would require a very large rewrite of the app's architecture, code and project structure, it would be a large task. The local SQLite database would have to be expanded to support storage of all a user's templates, workouts, etc. and all the data would have to be periodically synced to the backend whenever there is a good internet connection available.

Quality of Life: There are lots of minor features that would make the app experience better. Including but not limited to, a notification/vibration when your rest timer is over, adding exercises during a workout not just sets, pagination on the template and workout pages to save memory and reduce the size of API responses, and refresh buttons instead of pinging the API every time a new tab is focused on.

Cross Platform: React Native is also a multiplatform technology, so in theory this application could easily be ported to work with IOS. The only hurdle would be the different settings for lots of UI elements that are IOS specific, and implementing Apple social login which is a requirement for an IOS app if you support a Google login

9.4 Summary

I'm very proud of what I've managed to achieve in my time working on this project, the journey was both very difficult yet very rewarding; it has given me the skills to further my career and the confidence I need to pursue and tackle other projects like this in the future.

References

Steele, J., Fisher, J.P., Giessing, J., Androulakis-Korakakis, P., Wolf, M., Kroeske, B. and Reuters, R. (2022). Long-Term Time-Course of Strength Adaptation to Minimal Dose Resistance Training Through Retrospective Longitudinal Growth Modeling. *Research Quarterly for Exercise and Sport*, pp.1–18. doi:<https://doi.org/10.1080/02701367.2022.2070592>.

Aslam, S., Habyarimana, J.D.D. and Bin, S.Y. (2025). Neuromuscular adaptations to resistance training in elite versus recreational athletes. *Frontiers in Physiology*, 16. doi:<https://doi.org/10.3389/fphys.2025.1598149>.

Brzycki, Matthew. (1993). Strength Testing—Predicting a One-Rep Max from Reps-to-Fatigue. *Journal of Physical Education, Recreation & Dance*. 64. 88-90. 10.1080/07303084.1993.10606684.

Appendix A – Gen AI Tools Accountability Statement

ECS635U – Project Module			
Generative AI Tools – Student Accountability Statement			
The completed and signed statement must be included in the appendix of the progress and final reports.			
Student Name: Mohammed Hamza Huda			
Project Title: Lifting Log			
Supervisor Name: Nafi Ahmad			
1. Have you used Generative AI tools for your project?			
YES			
NO			
If yes, please complete sections 2 and 3 and sign the declaration below.			
If no, please sign the declaration below.			
2. Please provide details of all the AI tools that you used in your project work.			
AI Tool	Version	Publisher	Access URL
Gemini	2 Pro	Google	https://gemini.google.com
3. If you have used Artificial Intelligence (AI) tools in any part of your project please include in table format how, where, and why these tools were used. This should include references to the relevant parts of your project work — for example:			
- Specific chapters or sections of the report (e.g. literature review, methodology, requirements) - Software development (e.g. AI-assisted code generation, debugging, documentation) - Interview or survey materials (e.g. AI-generated or refined questions/prompts) - Datasets (e.g. AI-assisted data cleaning, labelling, or synthesis) - The showcase video (e.g. AI-assisted scripting, narration, or editing support)			
Be specific and transparent about your use of AI tools, including both direct outputs (e.g. text, code, images) and indirect uses (e.g. idea generation, editing assistance). The goal is to ensure academic integrity and provide a clear record of how AI contributed to your work.			
Project Section	Description	Justification	
Project Idea	Used AI to refine my initial idea after supervisor suggested I need a unique selling point for the app	I used GenAI to generate lots of different possible USPs in different fields of computer science that I then explored through research to decide on a final project idea.	
3. Generative AI Student Declaration			
- I have declared all uses of AI tools (e.g., coding assistants, text generators, data analysis tools) in this project. - I have reviewed and verified all AI-assisted content to ensure accuracy and suitability. - I take full responsibility for the originality and integrity of my project submission. - I have referenced all sources, including those suggested or produced with AI support. - I have not used AI in ways that undermine academic honesty (e.g., plagiarism, unverified code submission). - My use of AI was supplementary and did not replace my own independent learning and problem-solving.			
Signed: 		Date: 05/05/26	

Appendix B – Project Time Plan

TASK	START	END	DESCRIPTION	PHASE
Deliberation and deciding on a project	1/10/25	8/10/25	Look at possible project prospects, what they would entail and decide on a project	1
Research into technologies to be used/skills needed	8/10/25	13/10/25	Research into the frameworks and resources available for building mobile applications	1
Project Definition and Plan	13/10/25	22/10/25		1
Project Definition & Plan Deadline	22/10/25	23/10/25		DEADLINE
Update project idea to have a USP	23/10/25	27/10/25	Research and deliberate possible unique selling points for the application and update the project plan, accordingly, based on supervisor's feedback	2
Build core CRUD application	27/10/25	10/12/25	Implementing the basic functionality of the app without the "Invisible Coach" features and data analytics	2
Make progress presentation slides	10/12/25	15/12/25		2
Project Progress Presentation	15/12/25	16/12/25		DEADLINE
Write interim report	16/12/25	07/01/26		2
Progress Presentation and Report Submission	07/01/26	08/01/26		DEADLINE
Develop "Invisible Coach" features	08/01/26	08/03/26	Develop the data analytics tab, automatic progressive overload, strength goal setting and prediction	3
Develop tests	08/03/26	30/3/26	Unit testing to verify app functionality	3
Write final report first draft	08/01/26	30/3/26	Written throughout final development of the project	3
Draft Report Submission	30/3/26	31/3/26		DEADLINE
Final report based on feedback	31/3/26	5/5/26	Final write up with all screenshots of the app and code snippets	4
Testing, evaluating, bug fixes	31/3/26	30/4/26		4
Final report and supporting material submission	5/5/26	6/5/26		DEADLINE